My project mainly used AI in smaller prompts to help debug the code and add specific features. I made the face effects myself, but I used AI to help apply those effects to the actual diamond shape in the fragment shader. The effects on the diamond faces are inverted color, silver effect, distorted webcam image, blue hue effect, embossed effect, posterized color effect, warm effect, and glitch effect. Aside from this, I also created the procedural textures for the walls (including wood, metal, stone, and carpet-style textures). AI helped organize them into the dropdown menu and helped connect the uploaded wall image option to the shader. The floor uses a procedural stone texture, while the walls can be changed using the dropdown. I used AI to help with the "Lockdown" feature, which adds red laser beams to the scene to give it a diamond-in-a-vault feeling. AI also helped clean up the HTML user interface by arranging the sliders, buttons, dropdown, and file upload input.

Overall, I found that AI was most useful for connecting parts of the program together. For example, with the wall textures, I had wanted to map images that I added into the program folder onto the walls but was unable to do so due to security settings. AI was the one to suggest the upload image feature. However, I still had to understand and check the code because some early versions did not work correctly and even completely crashed my program. This was seen when I had initially prompted Chat GPT to take the coded face effects and transfer them to each diamond face. I had to continuously alter my prompts to fix this.

The application code and shaders interact through attributes and uniforms. The application creates arrays for vertex positions, normals, texture coordinates, and effect numbers. These are sent to the vertex shader through attributes (aPosition, aNormal, aTexCoord, and aEffectNumber). The vertex shader transforms each vertex using the model-view matrix and projection matrix. The model-view matrix places the object in the scene and controls its position and rotation. The projection matrix creates the perspective camera view. The normal matrix is also sent to the shader so that normals are transformed correctly for lighting. The vertex shader passes the transformed normal, position, texture coordinates, and effect number to the fragment shader. This was reused from my previous projects.

The webcam is used as a texture. In the JavaScript file, the program accesses the webcam through the hidden video element. During each frame, the current webcam frame is copied into a WebGL texture using gl.texImage2D. The fragment shader then samples this webcam texture using the texture coordinates. Each face of the diamond has an effect number, and the fragment shader uses that number to decide which image-processing effect to apply to that face.

The wall textures are handled in the fragment shader using a dropdown value from the application. The JavaScript sends the selected wall mode as the uniform uWallMode. Based on that value, the fragment shader chooses between wood, metal, stone, carpet, or an uploaded image texture. The floor is separate and always uses the procedural stone texture.

The lighting is handled in the fragment shader with a Phong-style lighting function. The shader uses the normal vector, light direction, eye direction, and halfway vector to calculate ambient, diffuse, and specular lighting. This makes the diamond and room look more three-dimensional instead of flat. The metallic face also uses extra specular lighting to make it look shinier.

Overall, I think the final code was pretty successful. Other than the minor changes that needed to be made to the AI-generated code, utilizing the tool was very helpful in connecting different ideas I had but didn't know how to implement. One limitation is that the uploaded image texture maps once onto each wall, which works well for most images but may not always look perfect depending on the picture chosen.